

Lesson: Binary Search Tree (BST) and Tree Abstract Data Type (ADT)

Overview of Tree ADT

A **Tree** is a hierarchical data structure that consists of nodes connected by edges. Each tree has a root node, and each node can have zero or more child nodes. Trees are widely used for various applications, including representing hierarchical data, searching, and sorting.

Key Concepts:

1. **Node:** The basic unit of a tree that contains data and links to its children.
2. **Root:** The top node of the tree.
3. **Leaf:** A node with no children.
4. **Subtree:** A tree formed by a node and its descendants.

Binary Search Tree (BST)

A **Binary Search Tree** is a special type of binary tree that maintains the following properties:

1. Each node has at most two children (left and right).
2. The left subtree of a node contains only nodes with keys less than the node's key.
3. The right subtree of a node contains only nodes with keys greater than the node's key.
4. Both left and right subtrees must also be binary search trees.

Operations on BST

1. **Insertion:** Add a new node while maintaining BST properties.
2. **Search:** Find a node in the tree.
3. **Deletion:** Remove a node and maintain the BST properties.
4. **Traversal:** Visit nodes in a specific order (e.g., inorder, preorder, postorder).

C++ Implementation of a Binary Search Tree

Let's implement a simple Binary Search Tree in C++ with the basic operations: insertion, search, and inorder traversal.

C++ Code Example

```
#include <iostream>
using namespace std;

// Node structure for the binary search tree
struct Node {
    int key;           // The value of the node
    Node* left;        // Pointer to the left child
    Node* right;       // Pointer to the right child

    // Constructor to create a new node
    Node(int value) {
        key = value;   // Initialize the key
        left = nullptr; // Initialize left child as null
        right = nullptr; // Initialize right child as null
    }
};

// Function to insert a new key into the BST
Node* insert(Node* root, int key) {
    // If the tree is empty, create a new node
    if (root == nullptr) {
        return new Node(key);
    }

    // Otherwise, recur down the tree
    if (key < root->key) {
        root->left = insert(root->left, key); // Insert in left subtree
    } else if (key > root->key) {
        root->right = insert(root->right, key); // Insert in right subtree
    }

    // Return the unchanged root pointer
}
```

```

        return root;
    }

    // Function to search for a key in the BST
    Node* search(Node* root, int key) {
        // Base cases: root is null or key is present at root
        if (root == nullptr || root->key == key) {
            return root;
        }

        // Key is greater than root's key, search in the right subtree
        if (key > root->key) {
            return search(root->right, key);
        }

        // Key is smaller than root's key, search in the left subtree
        return search(root->left, key);
    }

    // Function for inorder traversal of the BST
    void inorder(Node* root) {
        if (root != nullptr) {
            inorder(root->left);    // Visit left subtree
            cout << root->key << " "; // Visit node itself
            inorder(root->right);   // Visit right subtree
        }
    }

    // Main function to demonstrate the BST operations
    int main() {
        Node* root = nullptr; // Initialize the root of the BST

        // Inserting nodes into the BST
        root = insert(root, 50);
        root = insert(root, 30);
        root = insert(root, 70);
        root = insert(root, 20);
        root = insert(root, 40);
        root = insert(root, 60);
        root = insert(root, 80);
    }

```

```

// Inorder traversal of the BST
cout << "Inorder traversal of the BST: ";
inorder(root);
cout << endl;

// Searching for a key in the BST
int key = 40;
Node* result = search(root, key);
if (result != nullptr) {
    cout << "Key " << key << " found in the BST." << endl;
} else {
    cout << "Key " << key << " not found in the BST." << endl;
}

return 0; // Indicate successful completion
}

```

Explanation of the Code:

1. Node Structure:

- Defines a **Node** struct containing an integer **key**, and pointers to the left and right children.

2. Insert Function:

- Inserts a key into the BST. If the tree is empty, a new node is created. If the key is less than the current node's key, it recurses into the left subtree; otherwise, it goes into the right subtree.

3. Search Function:

- Searches for a key in the BST. It checks if the current node is null or matches the key. If not, it decides to search in either the left or right subtree based on the key's value.

4. Inorder Traversal:

- Traverses the tree in an inorder manner (left, root, right), which results in sorted output of the keys.

5. Main Function:

- Demonstrates the insertion of nodes and performs an inorder traversal. It also demonstrates searching for a specific key.

Conclusion

This lesson introduced the concepts of tree ADT and binary search trees. You learned how to implement a BST in C++ and perform basic operations like insertion, searching, and traversal. Understanding these concepts is crucial for efficient data management and retrieval in computer science.